



Curso LEIC

Programação Orientada a Objetos

Ano Letivo 2024 / 2025

Projeto:
NAUTILUS

Alunos:

14072, Afonso Pereira

14058, Dinis Carraça

14302, José Ribeiro

Grupo 3

Paço de Arcos, 20 de janeiro de 2025

Índice:

Introdução	2
Descrição dos Algoritmos.....	2
Resultados Obtidos.....	10
Discussão	16
Conclusões.....	17
Referências Bibliográficas.....	18

Introdução

O objetivo principal do projeto NAUTILUS é desenvolver um sistema que permita a gestão eficiente de uma frota náutica para um porto operacional marítimo. A aplicação será baseada nos princípios da Programação Orientada a Objetos (POO), utilizando a linguagem Java.

Visão Geral da Aplicação:

- A aplicação permite:
 - Gestão de marinheiros e embarcações.
 - Registro e consulta de missões realizadas.
 - Alocação e ativação de embarcações em zonas específicas (Norte, Sul, Este, Oeste).

Organização do Documento

- Descrição dos Algoritmos: Estruturas e funções principais.
- Resultados Obtidos: Outputs do sistema e suas relações com o código.
- Discussão: Análise dos resultados, problemas e soluções implementadas.
- Conclusões: Reflexão sobre os objetivos atingidos e propostas de melhorias.
- Referências Bibliográficas: Fontes consultadas.

Descrição dos Algoritmos

→ Marinheiro criarMarinheiro()

Propósito Geral: Criar um novo objeto da classe Marinheiro com os atributos fornecidos pelo utilizador.

Situações de Invocação: Invocada sempre que o utilizador deseja criar um novo marinheiro.

Parâmetros: Não possui.

Tipo de Retorno: Retorna um objeto Marinheiro.

Descrição do Algoritmo:

1. Solicita ao utilizador os seguintes dados:
 - Nome: Deve ser uma string sem números.
 - Patente: Deve pertencer ao enum PATENTE (Praça, Sargento ou Oficial). Se o utilizador escolher "Oficial", o sistema verifica se o marinheiro tem pelo menos 35 anos.
 - Data de nascimento: Deve estar no formato "aaaa-mm-dd".
2. Caso os dados fornecidos não sejam válidos, solicita novamente ao utilizador até obter entradas corretas.

3. Cria o objeto Marinheiro com os dados validados.
4. Adiciona o novo marinheiro à lista global de marinheiros.
5. Retorna o objeto criado.

→ **Embarcacao criarEmbarcacao()**

Propósito Geral: Criar um novo objeto da classe Embarcação com os atributos fornecidos pelo utilizador.

Situações de Invocação: Invocada sempre que o utilizador deseja criar uma nova embarcação.

Parâmetros: Não possui.

Tipo de Retorno: Retorna um objeto Embarcacao.

Descrição do Algoritmo:

1. Solicita ao utilizador os seguintes dados:
 - Nome: Obtido através da função String nomeUserGetter().
 - Marca e modelo: Inserção direta pelo utilizador.
 - Data de criação: No formato "aaaa-mm-dd".
 - Zona: Deve ser Norte, Sul, Este ou Oeste.
 - Estado de atracação: O utilizador insere 1 (atracado) ou 2 (não atracado). Repetir até uma entrada válida.
 - Tripulação: O utilizador escolhe adicionar ou não. Se optar por adicionar, a função void criarTripulacao() é chamada.
2. Cria o objeto Embarcacao com os dados fornecidos.
3. Adiciona o objeto criado à lista global de embarcações.
4. Retorna o objeto criado.

→ **String nomeUserGetter()**

Propósito Geral: Garantir que o nome da embarcação é único.

Situações de Invocação: Invocada durante a criação de embarcações para validar o nome.

Parâmetros: Não possui.

Tipo de Retorno: Retorna o nome da embarcação como string ou nada se o nome já existir.

Descrição do Algoritmo:

1. Solicita ao utilizador um nome.
2. Percorre a lista global de embarcações e verifica se o nome inserido já existe.
3. Caso o nome já esteja em uso, alerta o utilizador e solicita outro nome.
4. Retorna o nome inserido assim que for único.

→ **void criarTripulacao(ArrayList lista, String tipo)**

Propósito Geral: Adicionar marinheiros a uma tripulação de embarcação.

Situações de Invocação: Invocada durante a criação de embarcações para gerenciar a tripulação.

Parâmetros: Uma lista do tipo ArrayList<Marinheiro> e uma string tipo que representa a subclasse da embarcação

Tipo de Retorno: Não possui retorno.

Descrição do Algoritmo:

1. Solicita ao utilizador uma das opções:
 - Criar novos marinheiros.
 - Adicionar marinheiros existentes procurando pelo nome.
2. Solicita a quantidade de marinheiros a adicionar.
3. Dependendo da opção:
 - Se criar novos marinheiros, pede o número de marinheiros adicionar, este que não pode exceder um certo valor para cada tipo.
 - i. Chama void criarTripulante(lista) num loop até adicionar todos os marinheiros desejados.
 - Se adicionar existentes, chama Marinheiro procurarMarinheiro() para cada nome solicitado.
4. Adiciona os marinheiros à lista de tripulação da embarcação.

→ **Marinheiro procurarMarinheiro()**

Propósito Geral: Localizar um marinheiro existente na lista global.

Situações de Invocação: Invocada para localizar marinheiros existentes durante a criação de tripulações.

Parâmetros: Não possui.

Tipo de Retorno: Retorna o objeto Marinheiro encontrado ou null se não existir.

Descrição do Algoritmo:

1. Solicita ao utilizador o nome do marinheiro a procurar.
2. Percorre a lista global de marinheiros.
3. Caso encontre um marinheiro com o nome fornecido, retorna o objeto correspondente.
4. Se não encontrar, retorna null.

→ **void enviarMissoes(String tipo, ZONA zona)**

Propósito Geral: Enviar embarcações para missões de um tipo e zona específicos.

Situações de Invocação: Invocada quando o utilizador solicita o envio de embarcações para missões.

Parâmetros:

- String tipo: Tipo da missão (perseguição, apoio ou procura e resgate).
- ZONA zona: Zona da missão (Norte, Sul, Este ou Oeste).

Tipo de Retorno: Não possui retorno.

Descrição do Algoritmo:

1. Percorre a lista global de embarcações.
2. Verifica se a embarcação atende ao tipo e à zona fornecidos.
3. Se for esse o caso, define o atributo isOnMission da embarcação como true.

→ **verListaEmbarcacoes()**

Propósito Geral: Permitir ao utilizador visualizar a lista de embarcações no sistema, filtrando as mesmas por estado de atracação ou por zonas.

Situações de Invocação: Invocada quando o utilizador deseja consultar informações sobre as embarcações cadastradas no sistema.

Parâmetros: Não possui.

Tipo de Retorno: Não retorna nenhum valor diretamente. Exibe informações das embarcações no terminal.

Descrição do Algoritmo:

1. Verificação Inicial:

- Verifica se a lista global de embarcações está vazia. Se estiver, informa ao utilizador e termina a execução da função.

2. Iteração de Escolhas:

- Solicita ao utilizador escolher entre as seguintes opções:
 - 1: Mostrar lista de embarcações atracadas.
 - 2: Mostrar lista de embarcações não atracadas (com possibilidade de filtro por zona).
 - 3: Voltar ao menu anterior.
- Se a escolha for inválida, repete a solicitação.

3. Opção 1 - Lista de Embarcações Atracadas:

- Verifica se há embarcações atracadas na lista global.
 - Caso não existam, exibe uma mensagem e termina a opção.
- Exibe informações das embarcações atracadas utilizando a lista de embarcações de um porto.

4. Opção 2 - Lista de Embarcações Não Atracadas:

- Pergunta ao utilizador se deseja filtrar por zona.
 - Caso o utilizador escolha **filtrar por zona**, solicita a zona pretendida: Norte, Sul, Este, Oeste ou Indefinido.
 - Obtém a lista de embarcações pertencentes à zona escolhida (se existirem) e exibe as informações de cada uma.
 - Caso não existam embarcações na zona selecionada, informa ao utilizador.
 - Caso o utilizador opte por **não filtrar por zona**, exibe todas as embarcações ordenadas por tipo.

5. Opção 3 - Voltar:

- Exibe uma mensagem de retorno e termina a execução da função.

6. Mensagem de Opção Inválida:

- Caso o utilizador insira um valor fora das opções permitidas, exibe uma mensagem de erro.

→ **String convertMarinheirosToString(ArrayList<Marinheiro> lista)**

Propósito Geral: Converter a lista global dos marinheiros em uma string para salvar em um arquivo de texto.

Situações de Invocação: Quando é necessário salvar os dados da lista global de marinheiros num ficheiro.

Parâmetros:

- ArrayList<Marinheiro> lista: Lista de marinheiros a ser convertida.

Tipo de Retorno: Retorna uma string representando todos os marinheiros na lista.

Descrição do Algoritmo:

1. Inicializa uma variável que armazenará o texto final.
2. Percorre cada marinheiro na lista recebida como parâmetro.
3. Obtém os valores dos atributos de cada marinheiro e concatena em uma string.
4. Adiciona o texto concatenado à variável do texto final.
5. Após percorrer toda a lista, retorna a string com o texto final.

→ **String convertBarcosToString(ArrayList<Embarcacao> lista)**

Propósito Geral: Converter a lista global das embarcações em uma string para salvar em um arquivo de texto.

Situações de Invocação: Quando é necessário salvar os dados da lista global de embarcações em um ficheiro.

Parâmetros:

- ArrayList<Embarcacao> lista: Lista de embarcações a ser convertida.

Tipo de Retorno: Retorna uma string representando todas as embarcações na lista.

Descrição do Algoritmo:

1. Inicializa uma variável do tipo StringBuilder para armazenar o texto final.
2. Percorre cada embarcação na lista recebida como parâmetro.
3. Obtém os valores dos atributos de cada embarcação e adiciona ao StringBuilder.
4. Após percorrer toda a lista, retorna a string construída com o texto final.

→ **void convertStringToBarcos(String texto)**

Propósito Geral: Converter o texto carregado de um ficheiro para objetos do tipo embarcação.

Situações de Invocação: Quando o utilizador deseja carregar dados de embarcações de um ficheiro de texto.

Parâmetros:

- String texto: Texto com as informações das embarcações.

Tipo de Retorno: Não possui.

Descrição do Algoritmo:

1. Converte o texto recebido em uma lista de embarcações.
2. Para cada embarcação na lista, divide o texto nos atributos.
3. Separa cada atributo em chave (nome do atributo) e valor (conteúdo do atributo).
4. Cria um novo objeto do tipo embarcação usando os valores obtidos.
5. Adiciona o objeto criado à lista global de embarcações.

→ **void convertStringToMarinheiros(ArrayList<Marinheiro> lista, String texto)**

Propósito Geral: Converter o texto carregado referente a marinheiros para objetos da classe Marinheiro.

Situações de Invocação: Quando é necessário interpretar texto de um ficheiro e convertê-lo para objetos da classe Marinheiro.

Parâmetros:

- ArrayList<Marinheiro> lista: Lista de marinheiros a ser populada.
- String texto: Texto com as informações dos marinheiros.

Tipo de Retorno: Não possui.

Descrição do Algoritmo:

1. Percorre o texto recebido e separa os atributos de cada marinheiro.
2. Para cada atributo, divide em chave (nome do atributo) e valor (conteúdo do atributo).
3. Cria um novo objeto do tipo Marinheiro usando os valores obtidos.
4. Adiciona o objeto criado à lista global de marinheiros e à tripulação correspondente.

→ **void guardarFicheiro()**

Propósito Geral: Salvar todo o texto das listas globais em um ficheiro de texto.

Situações de Invocação: Quando o utilizador deseja salvar os dados das listas globais em um ficheiro, seja durante a execução ou ao final.

Parâmetros: Não possui.

Tipo de Retorno: Não possui.

Descrição do Algoritmo:

1. Solicita ao utilizador o nome do ficheiro.
2. Verifica se o ficheiro já existe na diretoria atual.
3. Caso exista, converte as listas em texto e salva no ficheiro.

4. Caso não exista, oferece a opção de criar um novo ficheiro.
5. Salva todo o texto gerado no ficheiro.

→ **void carregarFicheiro()**

Propósito Geral: Carregar informações de um ficheiro para as listas globais de embarcações e marinheiros.

Situações de Invocação: Quando o utilizador deseja carregar informações de um ficheiro existente.

Parâmetros: Não possui.

Tipo de Retorno: Não possui.

Descrição do Algoritmo:

1. Lê todo o texto do ficheiro para uma string.
2. Separa o texto em duas partes: informações das embarcações e informações dos marinheiros.
 - As informações das embarcações são delimitadas por “---embarcacao—” e “---embarcacao---end—”.
 - As informações dos marinheiros são delimitadas por “---marinheiro—” e “---marinheiro---end—”.
3. Chama as funções apropriadas para converter o texto em objetos e atualizar as listas globais.

Resultados Obtidos

O primeiro output que aparece é o da escolha do modo a utilizar como se pode ver na fig(1).

```
Menu:
1 -> Modo Manutencao
2 -> Modo Utilizacao
3 -> Encerrar o programa
Escolha:
```

Fig(1) - Menu Inicial

Se escolher a primeira opção é redirecionado para o modo Manutenção.

```
Modo Manuntenciaoc
1 -> Porto
2 -> Embarcacao
3 -> Marinheiro
4 -> Voltar
Escolha: 1
```

Fig(2) - Modo Manutenção

Ao escolher a primeira opção do menu é redirecionado para o menu com as funções do porto.

```
Menu Porto
1 -> Criar Porto
2 -> Editar Porto
3 -> Remover Porto
4 -> Voltar
Escolha: |
```

Fig(3) - Menu Porto

Se digitar 1 vai se deparar com um campo para inserir o nome do porto que está preste a criar.

```
Menu Porto
1 -> Criar Porto
2 -> Editar Porto
3 -> Remover Porto
4 -> Voltar
Escolha: 1
Nome do Porto: Novo Porto
Porto 'Novo Porto' criado com sucesso!
```

Figura(4) - Criar porto

Se quiser editar o porto, apenas se este já estiver criado, escolhe a opção 2, que irá levar ao seguinte menu.

```
Menu Edição Porto
1 -> Editar Nome
2 -> Editar Zona
3 -> Ligar/Desligar Radar
4 -> Voltar
Escolha: |
```

Figura(5) - Editar Porto

Se no menu da figura 2 optar por entrar no menu das embarcações o utilizador vai de encontro ao seguinte menu.

```
Menu Embarcacao
1 -> Criar Embarcacao
2 -> Editar Embarcacao
3 -> Remover Embarcacao
4 -> Voltar
Escolha: |
```

Figura(6) - Menu Embarcações

Quando decide criar uma embarcação é dada a opção de escolher o tipo como se consegue reparar na seguinte figura.

```
1 -> Embarcacao generica
2 -> Lancha Rapida
3 -> Navio Suporte
4 -> Barco Patrulha
5 -> Voltar
Escolha: |
```

Figura(7) - Tipos de Embarcações

Depois de escolhido o tipo o output fica da seguinte maneira.

```
1 -> Embarcacao generica
2 -> Lancha Rapida
3 -> Navio Suporte
4 -> Barco Patrulha
5 -> Voltar
Escolha: 1
      Nova embarcacao generica
Nome: Embarcação Genérica Teste
Marca: Phantom
Modelo: Phatom 200
Data (aaaa-mm-dd): 2010-10-10
Zona da Embarcação:
1 -> Norte
2 -> Sul
3 -> Este
4 -> Oeste
5 -> Indefinido
Escolha: 1
Criar Tripulacao (1 -> Sim / 2 -> Nao): 1
1 -> Criar Marinheiros
2 -> Adicionar por nome

Escolha: 1
Quantos marinheiros quer adicionar: 1
CRIAR MARINHEIRO:
Nome: Jonas Pistolas
Patente: SARGENTO
Data de nascimento (aaaa-mm-dd): 1990-10-10
```

Figura(8) - Criar Embarcação final

No caso de já ter uma embarcação criada e quiser alterar os atributos da mesma vai de encontro com a seguinte sequência de outputs.

```

      Modo Manutencao
1 -> Porto
2 -> Embarcacao
3 -> Marinheiro
4 -> Voltar
Escolha: 2

      Menu Embarcacao
1 -> Criar Embarcacao
2 -> Editar Embarcacao
3 -> Remover Embarcacao
4 -> Voltar
Escolha: 2
Quer ver a lista de embarcacoes?
1 - Sim, 2 - Nao: 2

Nome da embarcação a editar: Embarcação Genérica Teste

1 - Nome
2 - Marca
3 - Modelo
4 - Data
Atributo a mudar:3
Novo modelo: Phantom 500

```

```

      Menu Embarcacao
1 -> Criar Embarcacao
2 -> Editar Embarcacao
3 -> Remover Embarcacao
4 -> Voltar
Escolha: 2
Quer ver a lista de embarcacoes?
1 - Sim, 2 - Nao: 2

Nome da embarcação a editar: Embarcação Genérica Teste

1 - Nome
2 - Marca
3 - Modelo
4 - Data
Atributo a mudar:1
Novo nome: Embarcação Genérica

```

Fig (9) & (10) - Editar Embarcação

Na terceira opção do modo de utilização, todas as opções seguem um padrão semelhante ao descrito acima.

```

      Menu Marinheiro
1 -> Criar Marinheiro
2 -> Editar Marinheiro
3 -> Remover Marinheiro
4 -> Voltar
Escolha:

```

Figuras(11) - Menu marinheiros

Regressando ao menu da figura 1 se decidir desta vez utilizar o modo de utilização é redirecionado para o seguinte menu

```

      Modo Utilizacao
1 -> Enviar missão
2 -> Terminar missão
3 -> Ver listas
4 -> Ficheiros
5 -> Voltar
Escolha: |

```

Fig(12) - Modo Utilização

Se for a primeira opção a escolhida segue-se o seguinte output:

```

Tipos de missoes:
1 -> Perseguiçao e Captura
2 -> Apoio
3 -> Procura e Salvamento
4 -> Voltar
Escolha: |

```

Fig(13) - Menu Enviar Missões

Quando quiser ver as listas escolhe a segunda opção do menu da figura onde segue para o seguinte output:

```

Menu Listas
1 -> Lista Embarcacoes
2 -> Lista Marinheiros
3 -> Lista atributos
4 -> Voltar
Escolha:

```

Fig(14) - Menu ver listas

Se escolher uma das duas primeiras opções é gerado então o output das listas de embarcações e marinheiros, respectivamente.

```

Menu Listas
1 -> Lista Embarcacoes
2 -> Lista Marinheiros
3 -> Lista atributos
4 -> Voltar
Escolha: 1
Lista de embarcações atracadas? (1 -> Sim / 2 -> Nao / 3 -> Voltar): 2
Lista de embarcações por zona? (1 -> Sim / 2 -> Não): 2
Lista de todas as embarcações:
=====
INFO EMBARCACAO
=====
Nome      : Lancha Rápida Teste 1
Marca     : Phantom
Modelo    : Phantom 300
ID        : 1000
Marca     : Phantom
Zona      : NORTE
Tipo      : Lancha Rapida
Atracado  : Afirmativo
Em missao : Negativo
Tripulacao:
=====

```

```

Menu Listas
1 -> Lista Embarcacoes
2 -> Lista Marinheiros
3 -> Lista atributos
4 -> Voltar
Escolha: 2
Lista de marinheiros:
=====
INFO MARINHEIRO
=====
Nome:      Jonas
Data de Nascimento: 1980-06-06
Patente:   OFICIAL
ID:        1000
=====
INFO MARINHEIRO
=====
Nome:      Pedro
Data de Nascimento: 1999-10-10
Patente:   SARGENTO
ID:        1001
=====

```

Fig(15) e Fig(16) -Listas de Embarcações e Marinheiros

Ao escolher a terceira opção tem de a seguir decidir qual a lista que quer ordenada.

```

Lista por atributos
1 -> Marinheiros
2 -> Embarcacoes
3 -> Voltar
Escolha:

```

Fig(17) - Listas por atributos

Depois de seleccionar por qual atributo quer ordenar o output que se segue é o mesmo que nas figuras 15 e 16.

<pre> Lista por atributos 1 -> Marinheiros 2 -> Embarcacoes 3 -> Voltar Escolha: 1 Atributo: 1 -> Id 2 -> Nome 3 -> Patente 4 ->Data de Nascimento 5 -> Voltar Escolha: 1 Lista por id: </pre>	<pre> ===== INFO MARINHEIRO ===== Nome: Jonas Data de Nascimento: 1980-06-06 Patente: OFICIAL ID: 1000 ===== INFO MARINHEIRO ===== Nome: Pedro Data de Nascimento: 1999-10-10 Patente: SARGENTO ID: 1001 ===== </pre>
--	---

Fig (18) & (19) - Lista Marinheiros por atributo

<pre> Escolha: 1 Lista ordenada por id: ===== INFO EMBARCACAO ===== Nome : Barco muito rápido Marca : Phantom Modelo : Phantom 300 ID : 1000 Marca : Phantom Zona : NORTE Tipo : Generica Atracado : Afirmativo Em missao : Negativo Tripulacao: </pre>	<pre> ===== INFO EMBARCACAO ===== Nome : Lancha Rápida Louca Marca : Phantom Modelo : Phantom 500 ID : 1001 Marca : Phantom Zona : SUL Tipo : Lancha Rapida Atracado : Afirmativo Em missao : Negativo Tripulacao: ===== INFO MARINHEIRO ===== Nome: Jonas Data de Nascimento: 1980-06-06 Patente: OFICIAL ID: 1000 ===== </pre>
--	--

Fig(20) & (21) - Lista Embarcação por atributo

Regressando ao menu da figura 12 se decidir entrar na quarta opção vai ser redireccionado para o menu dos ficheiros que tem a seguinte disposição:

```

Menu Ficheiros
1 -> Guardar informacoes
2 -> Carregar informacoes
3 -> Exportar lista de embarcacoes
4 -> Voltar
Escolha:

```

Figura(22) - Menu ficheiros

O output para as duas primeiras opções é idêntico ao que se pode ver na seguinte figura.


```
Menu Ficheiros
1 -> Guardar informacoes
2 -> Carregar informacoes
3 -> Exportar lista de embarcacoes
4 -> Voltar
Escolha: 1
Nome do ficheiro: info.txt
Conteudo guardado com sucesso no ficheiro: info.txt
```

Figura(23) - Nomear ficheiros

Se o ficheiro não existir surge um novo output para o criar.

```
Menu Ficheiros
1 -> Guardar informacoes
2 -> Carregar informacoes
3 -> Exportar lista de embarcacoes
4 -> Voltar
Escolha: 2
Nome do ficheiro: novo
Ficheiro inesistente na directoria atual.
Quer criar o ficheiro?
1 -> Sim
2 -> No
Escolha: 1
Nome do ficheiro: novo
A criar ficheiro: novo.txt
Ficheiro criado com sucesso!
```

Figura(24) - Criar ficheiros

Discussão

Os resultados obtidos nos testes ao programa foram coerentes com os resultados esperados. Os testes às operações realizadas pelo sistema demonstraram que os principais objetivos, como a gestão do porto, marinheiros e embarcações, foram atingidos.

Coerência dos Resultados com os objetivos Esperados:

- **Gestão do Porto, Marinheiros e Embarcações:**

As funcionalidades relacionadas à criação, edição e consulta do porto, marinheiros e embarcações funcionaram corretamente.

- **Criação de ficheiros e guardar informações nos mesmos:**

O programa consegue, com brio, criar um ficheiro e transferir toda a informação presente nas listas, de modo a que o utilizador consiga-a guardar localmente.

Problemas Encontrados e Soluções Implementadas:

1. **Dificuldade em Relacionar Objetos de Diferentes Classes:**

Durante o desenvolvimento, foi desafiador estabelecer relações adequadas entre os objetos das diferentes classes.

Solução: Criação da classe DadosGerais que carrega informações vitais para o correto funcionamento do programa.

2. **Carregamento/Gravação de Dados em Ficheiros:**

A conversão das informações, principalmente do ficheiro para o programa, provou ser uma das tarefas mais trabalhosas pois, por vezes, o que é feito não é tangível, logo, torna-se mais difícil descobrir onde o erro foi cometido.

Soluções:

Programa → Ficheiro : Converter todos os objetos contidos nas listas em strings e concatená-las todas de modo a formar o texto completo.

Ficheiro → Programa : Converter o texto em objetos que são depois adicionados às respectivas listas.

3. **Tamanho e Complexidade do Projeto:**

O crescimento do projeto ao longo do tempo tornou o código difícil de navegar e testar em alguns momentos.

Solução:

A organização em menus hierárquicos, com opções claras e fluxos de navegação, ajudou a modularizar a interação do utilizador.

Conclusões

- Conclusões da ‘Discussão’ de resultados:
 - Encontrámos soluções, mesmo que menos ideais, para resolver os problemas.
- Objetivos Atingidos:
 - Gestão completa de marinheiros e embarcações.
 - Implementação de um sistema Radar
- Objetivos parcialmente atingidos:
 - Tratamento de erros com exceções
 - Gestão de ficheiros, especificamente, o ficheiro só pode ser usado uma vez se for carregado devido a funcionar com `append()` ou seja apenas acrescenta informação e não da `override`.
- Objetivos não atingidos:
 - Verificação das patentes da tripulação no ato da criação da mesma.
 - Certificação de id's não duplicados
- Desenvolvimentos futuros:
 - Adicionar interface gráfica.
 - Permitir ao utilizador escolher o formato em que quer guardar o ficheiro, .doc ou .pdf por exemplo.
 - Dar a opção ao utilizador de, quando decide guardar um ficheiro, se quer dar `append` ou `overwrite` ao mesmo

Referências Bibliográficas

W3SCHOOLS

<https://www.w3schools.com/>

PowerPoints 8, 9 e 11

Unidade Curricular de Programação Orientada a Objetos

Prof. Gonçalo Piedade